Interview Questions About Verilog

1) Write a verilog code to swap contents of two registers with and without a temporary register?

With temp reg ;

```
always @ (posedge clock)
begin
temp=b;
b=a;
a=temp;
end
```

Without temp reg;

```
always @ (posedge clock)
begin
a <= b;
b <= a;
end
```

How to write FSM is verilog?

there r mainly 4 ways 2 write fsm code
1) using 1 process where all input decoder, present state, and output decoder r combine in one process.
2) using 2 process where all comb ckt and sequential ckt separated in different process
3) using 2 process where input decoder and persent state r combine and output decoder seperated in other process
4) using 3 process where all three, input decoder, present state and output decoder r separated in 3 process.

What is difference between freeze deposit and force?

$deposit(variable, value);
This system task sets a Verilog register or net to the specified value. variable is the register or net to be changed; value is the new value for the register or net. The value remains until there is a subsequent driver transaction or another $deposit task for the same register or net. This system task operates identically to the ModelSim force -deposit command.

The force command has -freeze, -drive, and -deposit options. When none of these is specified, then -freeze is assumed for unresolved signals and -drive is assumed for resolved signals. This is designed to provide compatibility with force files. But if you prefer -freeze as the default for both resolved and unresolved signals.

Will case infer priority register if yes how give an example?
yes case can infer priority register depending on coding style

```
reg r;
// Priority encoded mux,
always @ (a or b or c or select2)
begin
r = c;
case (select2)
2'b00: r = a;
2'b01: r = b;
endcase
```

end

Casex,z difference,which is preferable,why?

CASEZ :
Special version of the case statement which uses a Z logic value to represent don't-care bits. CASEX : Special version of the case statement which uses Z or X logic values to represent don't-care bits.

CASEZ should be used for case statements with wildcard don't cares, otherwise use of CASE is required; CASEX should never be used.
This is because:
Don't cares are not allowed in the "case" statement. Therefore casex or casez are required. Casex will automatically match any x or z with anything in the case statement. Casez will only match z's -- x's require an absolute match.

Verilog interview Questions
Given the following Verilog code, what value of "a" is displayed?

```
always @(clk) begin
a = 0;
a <= 1;
$display(a);
end
```

This is a tricky one! Verilog scheduling semantics basically imply a
four-level deep queue for the current simulation time:
1: Active Events (blocking statements)
2: Inactive Events (#0 delays, etc)
3: Non-Blocking Assign Updates (non-blocking statements)
4: Monitor Events ($display, $monitor, etc).
Since the "a = 0" is an active event, it is scheduled into the 1st "queue".
The "a <= 1" is a non-blocking event, so it's placed into the 3rd queue.
Finally, the display statement is placed into the 4th queue. Only events in the active queue are completed this sim cycle, so the "a = 0" happens, and then the display shows a = 0. If we were to look at the value of a in the next sim cycle, it would show 1.

What is the difference between the following two lines of Verilog code?

```
#5 a = b;
a = #5 b;
```

#5 a = b; Wait five time units before doing the action for "a = b;".
a = #5 b; The value of b is calculated and stored in an internal temp register,After five time units, assign this stored value to a.

What is the difference between:

```
c = foo ? a : b;
and
if (foo) c = a;
else c = b;
```

The ? merges answers if the condition is "x", so for instance if foo = 1'bx, a = 'b10, and b = 'b11, you'd get c = 'b1x. On the other hand, if treats Xs or Zs as FALSE, so you'd always get c = b.

What are Intertial and Transport Delays ??

What does `timescale 1 ns/ 1 ps signify in a verilog code?

'timescale directive is a compiler directive.It is used to measure simulation time or delay time. Usage : `timescale / reference_time_unit : Specifies the unit of measurement for times and delays. time_precision: specifies the precision to which the delays are rounded off.

What is the difference between === and == ?

output of "==" can be 1, 0 or X.
output of "===" can only be 0 or 1.
When you are comparing 2 nos using "==" and if one/both the numbers have one or more bits as "x" then the output would be "X" . But if use "===" outpout would be 0 or 1.
e.g A = 3'b1x0
B = 3'b10x
A == B will give X as output.
A === B will give 0 as output.
"==" is used for comparison of only 1's and 0's .It can't compare Xs. If any bit of the input is X output will be X
"===" is used for comparison of X also.

How to generate sine wav using verilog coding style?

A: The easiest and efficient way to generate sine wave is using CORDIC Algorithm.

What is the difference between wire and reg?

Net types: (wire,tri)Physical connection between structural elements. Value assigned by a continuous assignment or a gate output. Register type: (reg, integer, time, real, real time) represents abstract data storage element. Assigned values only within an always statement or an initial statement. The main difference between wire and reg is wire cannot hold (store) the value when there no connection between a and b like a->b, if there is no connection in a and b, wire loose value. But reg can hold the value even if there in no connection. Default values:wire is Z,reg is x.

How do you implement the bi-directional ports in Verilog HDL?

```
module bidirec (oe, clk, inp, outp, bidir);

// Port Declaration
input oe;
input clk;
input [7:0] inp;
output [7:0] outp;
inout [7:0] bidir;
reg [7:0] a;
reg [7:0] b;
assign bidir = oe ? a : 8'bZ ;
assign outp = b;
// Always Construct
always @ (posedge clk)
begin
b <= bidir;
a <= inp;
end
endmodule
```

what is verilog case (1) ?

```
wire [3:0] x;
always @(...) begin
case (1'b1)
x[0]: SOMETHING1;
x[1]: SOMETHING2;
x[2]: SOMETHING3;
x[3]: SOMETHING4;
endcase
end
```

The case statement walks down the list of cases and executes the first one that matches. So here, if the lowest 1-bit of x is bit 2, then something3 is the statement that will get executed (or selected by the logic).

Why is it that "if (2'b01 & 2'b10)..." doesn't run the true case?

This is a popular coding error. You used the bit wise AND operator (&) where you meant to use the logical AND operator (&&).

What are Different types of Verilog Simulators ?

There are mainly two types of simulators available.

Event Driven
Cycle Based

Event-based Simulator:

This Digital Logic Simulation method sacrifices performance for rich functionality: every active signal is calculated for every device it propagates through during a clock cycle. Full Event-based simulators support 4-28 states; simulation of Behavioral HDL, RTL HDL, gate, and transistor representations; full timing calculations for all devices; and the full HDL standard. Event-based simulators are like a Swiss Army knife with many different features but none are particularly fast.

Cycle Based Simulator:

This is a Digital Logic Simulation method that eliminates unnecessary calculations to achieve huge performance gains in verifying Boolean logic:

1.) Results are only examined at the end of every clock cycle; and
2.) The digital logic is the only part of the design simulated (no timing calculations). By limiting the calculations, Cycle based Simulators can provide huge increases in performance over conventional Event-based simulators.
Cycle based simulators are more like a high speed electric carving knife in comparison because they focus on a subset of the biggest problem: logic verification.
Cycle based simulators are almost invariably used along with Static Timing verifier to compensate for the lost timing information coverage.

What is Constrained-Random Verification ?

Introduction

As ASIC and system-on-chip (SoC) designs continue to increase in size and complexity, there is an equal or greater increase in the size of the verification effort required to achieve functional coverage goals. This has created a trend in RTL verification techniques to employ constrained-random verification, which shifts the emphasis from hand-authored tests to utilization of compute resources. With the corresponding emergence of faster, more complex bus standards to handle the massive volume of data traffic there has

also been a renewed significance for verification IP to speed the time taken to develop advanced testbench environments that include randomization of bus traffic.

Directed-Test Methodology

Building a directed verification environment with a comprehensive set of directed tests is extremely time-consuming and difficult. Since directed tests only cover conditions that have been anticipated by the verification team, they do a poor job of covering corner cases. This can lead to costly re-spins or, worse still, missed market windows.

Traditionally verification IP works in a directed-test environment by acting on specific testbench commands such as read, write or burst to generate transactions for whichever protocol is being tested. This directed traffic is used to verify that an interface behaves as expected in response to valid transactions and error conditions. The drawback is that, in this directed methodology, the task of writing the command code and checking the responses across the full breadth of a protocol is an overwhelming task. The verification team frequently runs out of time before a mandated tape-out date, leading to poorly tested interfaces. However, the bigger issue is that directed tests only test for predicted behavior and it is typically the unforeseen that trips up design teams and leads to extremely costly bugs found in silicon.

Constrained-Random Verification Methodology

The advent of constrained-random verification gives verification engineers an effective method to achieve coverage goals faster and also help find corner-case problems. It shifts the emphasis from writing an enormous number of directed tests to writing a smaller set of constrained-random scenarios that let the compute resources do the work. Coverage goals are achieved not by the sheer weight of manual labor required to hand-write directed tests but by the number of processors that can be utilized to run random seeds. This significantly reduces the time required to achieve the coverage goals.

Scoreboards are used to verify that data has successfully reached its destination, while monitors snoop the interfaces to provide coverage information. New or revised constraints focus verification on the uncovered parts of the design under test. As verification progresses, the simulation tool identifies the best seeds, which are then retained as regression tests to create a set of scenarios, constraints, and seeds that provide high coverage of the design.

What are the differences between blocking and nonblocking assignments?

While both blocking and nonblocking assignments are procedural assignments, they differ in behaviour with respect to simulation and logic synthesis as follows:

Blocking assignments	Nonblocking assignments
In a blocking assignment, the evaluation of the expression on the RHS is updated to the LHS variable autonomously based on the delay value (either 0 if no delay specified, or scheduled as a future event if a non-0 value is specified)	Nonblocking assignment to LHS is *scheduled* to occur when the next evaluation cycle occurs in simulation and not immediately. Updates are not available immediately within the same time unit
When multiple blocking assignments are present in a process, the trailing assignments are blocked from occurring until the current assignment is completed	Multiple nonblocking assignments can be scheduled to occur concurrently on the next evaluation cycle in simulation
There is a possibility of race conditions on the variables of blocking assignments if assignments happen to it from two processes concurrently	The race conditions are avoided as the updated value is assigned after evaluation
Recommended to use within combinatorial **always** blocks	Recommended to use within the sequential **always** blocks
Can be used in procedural assignments like ***initial***, ***always*** and continuous assignments to nets like ***assign*** statements	Can be used only in the procedural blocks like ***initial*** and ***always***; Continuous assignment to nets like the ***assign*** statement is not permitted
Represented by "=" operator sign between LHS and RHS	Represented by "<=" operator sign between LHS and RHS

How can I model a bi-directional net with assignments influencing both source and destination?

The assign statement constitutes a continuous assignment. The changes on the RHS of the statement immediately reflect on the LHS net. However, any changes on the LHS don't get reflected on the RHS. For example, in the following statement, changes to the rhs net will update the lhs net, but not vice versa.

System Verilog has introduced a keyword alias, which can be used only on nets to have a two-way assignment. For example, in the following code, any changes to the rhs is reflected to the lh s , and vice versa.

```
wire rhs , lhs
assign lhs=rhs;
```

System Verilog has introduced a keyword alias, which can be used only on nets to have a two-way assignment. For example, in the following code, any changes to the rhs is reflected to the lh s , and vice versa.

```
module test ();
wire rhs,lhs;
```

alias lhs=rhs;

In the above example, any change to either side of the net gets reflected on the other side.

Are tasks and functions re-entrant, and how are they different from static task and function calls?

In Verilog-95, tasks and functions were not re-entrant. From Verilog version 2001 onwards, the tasks and functions are reentrant. The reentrant tasks have a keyword automatic between the keyword task and the name of the task. The presence of the keyword automatic replicates and allocates the variables within a task dynamically for each task entry during concurrent task calls, i.e., the values don't get overwritten for each task call. Without the keyword, the variables are allocated statically, which means these variables are shared across different task calls, and can hence get overwritten by each task call.

Reentrant task	Static task
Has the keyword ***automatic*** between the ***task*** keyword and identifier	*Doesn't* have the keyword ***automatic*** between the ***task*** keyword and the identifier
Variables declared within the task are allocated dynamically for each concurrent task call	Variable declarations within the task are allocated statically
All variables will be replicated in each concurrent call to store state specific to that invocation	Each concurrent call to the task will OVERWRITE the statically allocated local variables of the task from all other concurrent calls to the task
Variables declared are de-allocated at the end of task invocation	Variables retain their values between invocations
Task items cannot be accessed by hierarchical inferences	Task items can be accessed by hierarchical inferences
Task items shall be allocated new across all uses of the task executing concurrently	Task items can be shared across all uses of the task executing concurrently

How can I override variables in an automatic task?

By default, all variables in a module are static, i.e., these variables will be replicated for all instances of a module. However, in the case of task and function, either the task/function itself or the variables within them can be defined as static or automatic. The following explains the inferences through different combinations of the task/function and/or its variables, declared either as static or automatic:

No automatic definition of task/function or its variables This is the Verilog-1995 format, wherein the task/function and its variables were implicitly static. The variables are allocated only once. Without the mention of the automatic keyword, multiple calls to task/function will override their variables.

static task/function definition

System Verilog introduced the keyword static. When a task/function is explicitly defined as static, then its variables are allocated only once, and can be overridden. This scenario is exactly the same scenario as before.

automatic task/function definition

From Verilog-2001 onwards, and included within SystemVerilog, when the task/function is declared as automatic, its variables are also implicitly automatic. Hence, during multiple calls of the task/function, the variables are allocated each time and replicated without any overwrites.

static task/function and automatic variables

SystemVerilog also allows the use of automatic variables in a static task/function. Those without any changes to automatic variables will remain implicitly static. This will be useful in scenarios wherein the implicit static variables need to be initialised before the task call, and the automatic variables can be allocated each time.

automatic task/function and static variables

SystemVerilog also allows the use of static variables in an automatic task/function. Those without any changes to static variables will remain implicitly automatic. This will be useful in scenarios wherein the static variables need to be updated for each call, whereas the rest can be allocated each time.

What are the rules governing usage of a Verilog function?

The following rules govern the usage of a Verilog function construct:

A function cannot advance simulation-time, using constructs like #, @. etc.
A function shall not have nonblocking assignments.
A function without a range defaults to a one bit reg for the return value.
It is illegal to declare another object with the same name as the function in the scope where the function is declared.

How do I prevent selected parameters of a module from being overridden during instantiation?

If a particular parameter within a module should be prevented from being overridden, then it should be declared using the localparam construct, rather than the parameter construct. The localparam construct has been introduced from Verilog-2001. Note that a localparam variable is fully identical to being defined as a parameter, too. In the following example, the localparam construct is used to specify num_bits, and hence trying to override it directly gives an error message.

```
module localparam_list (addr, data);
parameter width = 32;
parameter depth = 64;
localparam num_bits = width * depth;
input  [width-1 : 0] addr;
input  [depth-1 : 0] data;
...
endmodule
```

Note, however, that, since the width and depth are specified using the parameter construct, they can be overridden during instantiation or using defparam, and hence will indirectly override the num_bits values. In general, localparam constructs are useful in defining new and localized identifiers whose values are derived from regular parameters.

What are the pros and cons of specifying the parameters using the defparam construct vs. specifying during instantiation?

The advantages of specifying parameters during instantiation method are:

All the values to all the parameters don't need to be specified. Only those parameters that are assigned the new values need to be specified. The unspecified parameters will retain their default values specified within its module definition.

The order of specifying the parameter is not relevant anymore, since the parameters are directly specified and linked by their name.

The disadvantage of specifying parameter during instantiation are:

This has a lower precedence when compared to assigning using defparam.

The advantages of specifying parameter assignments using defparam are:

This method always has precedence over specifying parameters during instantiation.

All the parameter value override assignments can be grouped inside one module and together in one place, typically in the top-level testbench itself.

When multiple defparams for a single parameter are specified, the parameter takes the value of the last defparam statement encountered in the source if, and only if, the multiple defparam's are in the same file. If there are defparam's in different files that override the same parameter, the final value of the parameter is indeterminate.

The disadvantages of specifying parameter assignments using defparam are:

The parameter is typically specified by the scope of the hierarchies underneath which it exists. If a particular module gets ungrouped in its hierarchy, [sometimes necessary during synthesis], then the scope to specify the parameter is lost, and is unspecified. B

For example, if a module is instantiated in a simulation testbench, and its internal parameters are then overridden using hierarchical defparam constructs (For example, defparam U1.U_fifo.width = 32;). Later, when this module is synthesized, the internal hierarchy within U1 may no longer exist in the gate-level netlist, depending upon the synthesis strategy chosen. Therefore post-synthesis simulation will fail on the hierarchical defparam override.

Can there be full or partial no-connects to a multi-bit port of a module during its instantiation?

No. There cannot be full or partial no-connects to a multi-bit port of a module during instantiation

What happens to the logic after synthesis, that is driving an unconnected output port that is left open (, that is, noconnect) during its module instantiation?

An unconnected output port in simulation will drive a value, but this value does not propagate to any other logic. In synthesis, the cone of any combinatorial logic that drives the unconnected output will get optimized away during boundary optimisation, that is, optimization by synthesis tools across hierarchical boundaries.

How is the connectivity established in Verilog when connecting wires of different widths?

When connecting wires or ports of different widths, the connections are right-justified, that is, the rightmost bit on the RHS gets connected to the rightmost bit of the LHS and so on, until the MSB of either of the net is reached.

Can I use a Verilog function to define the width of a multi-bit port, wire, or reg type?

The width elements of ports, wire or reg declarations require a constant in both MSB and LSB. Before Verilog 2001, it is a syntax error to specify a function call to evaluate the value of these widths. For example, the following code is erroneous before Verilog 2001 version.

reg [port1(val1:vla2) : port2 (val3:val4)] reg1;

In the above example, get_high and get_low are both function calls of evaluating a constant result for MSB and LSB respectively. However, Verilog-2001 allows the use of a function call to evaluate the MSB or LSB of a width declaration

What is the implication of a combinatorial feedback loops in design testability?

The presence of feedback loops should be avoided at any stage of the design, by periodically checking for it, using the lint or synthesis tools. The presence of the feedback loop causes races and hazards in the design, and 104 RTL Design
leads to unpredictable logic behavior. Since the loops are delay-dependent, they cannot be tested with any ATPG algorithm. Hence, combinatorial loops should be avoided in the logic.

What are the various methods to contain power during RTL coding?

Any switching activity in a CMOS circuit creates a momentary current flow from VDD to GND during logic transition, when both N and P type transistors are ON, and, hence, increases power consumption.
The most common storage element in the designs being the synchronous FF, its output can change whenever its data input toggles, and the clock triggers. Hence, if these two elements can be asserted in a controlled fashion, so that the data is presented to the D input of the FF only when required, and the clock is also triggered only when required, then it will reduce the switching activity, and, automatically the power.

The following bullets summarize a few mechanisms to reduce the power consumption:

- Reduce switching of the data input to the Flip-Flops.
- Reduce the clock switching of the Flip-Flops.
- Have area reduction techniques within the chip, since the number of gates/Flip-Flops that toggle can be reduced.

How do I model Analog and Mixed-Signal blocks in Verilog?

First, this is a big area.Analog and Mixed-Signal designers use tools like Spice to fully characterize and model their designs.My only involvement with Mixed-Signal blocks has been to utilize behavioral models of things like PLLs, A/Ds, D/As within a larger SoC.There are some specific Verilog tricks to this which is what this FAQ is about (I do not wish to trivialize true Mixed-Signal methodology, but us chip-level folks need to know this trick).
A mixed-signal behavioral model might model the digital and analog input/output behavior of, for example, a D/A (Digital to Analog Converter).So, digital input in and analog voltage out.Things to model might be the timing (say, the D/A utilizes an internal Success Approximation algorithm), output range based on power supply voltages, voltage biases, etc.A behavioral model may not have any knowledge of the physical layout and therefore may not offer any fidelity whatsoever in terms of noise, interface, cross-talk, etc.A model **might** be parameterized given a specific characterization for a block.Be very careful about the assumptions and limitations of the model!
Issue #1; how do we model analog voltages in Verilog.Answer: use the Verilog real data type, declare “analog wires” as wire[63:0] in order to use a 64-bit floating-type represenation, and use the built-in PLI functions:
$rtoi converts reals to integers w/truncation e.g. 123.45 -> 123
$itor converts integers to reals e.g. 123 -> 123.0
$realtobits converts reals to 64-bit vector
$bitstoreal converts bit pattern to real
That was a lot.This is a trick to be used in vanilla Verilog.The 64-bit wire is simply a ways to actually interface to the ports of the mixed-signal block.In other words, our example D/A module may have an output called AOUT which is a voltage.Verilog does not allow us to declare an output port of type REAL.So, instead declare AOUT like this:

```
module dtoa (clk, reset..... aout.....);
....
wire [63:0]aout;// Analog output
....
```

We use 64 bits because we can use floating-point numbers to represent out voltage output (e.g. 1.22x10-3 for 1.22 millivolts).The floating-point value is relevant only to Verilog and your workstation and processor, and the IEEE floating-point format has NOTHING to do with the D/A implementation.Note the disconnect in terms of the netlist itself.The physical “netlist” that you might see in GDS may have a single metal interconnect that is AOUT, and obviously NOT 64 metal wires.Again, this is a trick.The 64-bit bus is only for wiring.You may have to do some quick netlist substitutions when you hand off a netlist.
In Verilog, the **real** data type is basically a floating-point number (e.g. like double in C).If you want to model an analog value either within the mixed-signal behavorial model, or externally in the system testbench (e.g. the sensor or actuator), use the real data type.You can convert back and forth between

real and your wire [63:0] using the PLI functions listed above.A trivial D/A model could simply take the digital input value, convert it to real, scale it according to some #defines, and output the value on AOUT as the 64-bit “psuedo-analog” value.Your testbench can then do the reverse and print out the value, or whatever.More sophisticated models can model the Successive Approximation algorithm, employ look-ups, equations, etc. etc.
That’s it.If you are getting a mixed-signal block from a vendor, then you may also receive (or you should ask for) the behavioral Verilog models for the IP.

How do I synthesize Verilog into gates with Synopsys?

The answer can, of course, occupy several lifetimes to completely answer.. BUT.. a straight-forward Verilog module can be very easily synthesized using Design Compiler (e.g. dc_shell). Most ASIC projects will create very elaborate synthesis scripts, CSH scripts, Makefiles, etc. This is all important in order automate the process and generalize the synthesis methodology for an ASIC project or an organization. BUT don't let this stop you from creating your own simple dc_shell experiments!
Let's say you create a Verilog module named foo.v that has a single clock input named 'clk'. You want to synthesize it so that you know it is synthesizable, know how big it is, how fast it is, etc. etc. Try this:

```
target_library = { CORELIB.db } <--- This part you need to get from your vendor...
read -format verilog foo.v
create_clock -name clk -period 37.0
set_clock_skew -uncertainty 0.4 clk
set_input_delay 1.0 -clock clk all_inputs() - clk - reset
set_output_delay 1.0 -clock clk all_outputs()
compile
report_area
report_timing
write -format db -hierarchy -output foo.db
write -format verilog -hierarchy -output foo.vg
quit
```

You can enter all this in interactively, or put it into a file called 'synth_foo.scr' and then enter:

```
dc_shell -f synth_foo.scr
```

You can spend your life learning more and more Synopsys and synthesis-related commands and techniques, but don't be afraid to begin using these simple commands.

How can I pass parameters to my simulation?

A testbench and simulation will likely need many different parameters and settings for different sorts of tests and conditions. It is definitely a good idea to concentrate on a single testbench file that is parameterized, rather than create a dozen seperate, yet nearly identical, testbenches. Here are 3 common techniques:

- Use a **define**. This is almost exactly the same approach as the #define and -D compiler arg that C programs use. In your Verilog code, use a `define to define the variable condition and then use the Verilog preprocessor directives like `ifdef. Use the '+define+' Verilog command line option. For example:

... to run the simulation ..

```
verilog testbench.v cpu.v +define+USEWCSDF
... in your code ...
`ifdef USEWCSDF
initial $sdf_annotate (testbench.cpu, "cpuwc.sdf");
`endif
```

The +define+ can also be filled in from your Makefile invocation, which in turn, can be finally

filled in the your UNIX promp command line.
Defines are a blunt weapon because they are very global and you can only do so much with them since they are a pre-processor trick. Consider the next approach before resorting to defines.

- Use **parameters** and parameter definition modules. Parameters are not preprocessor definitions and they have *scope* (e.g. parameters are associated with specific modules). Parameters are therefore more clean, and if you are in the habit of using a lot of defines; consider switching to parameters. As an example, lets say we have a test (e.g. test12) which needs many parameters to have particular settings. In your code, you might have this sort of stuff:

```
module testbench_uart1 (....)
parameter BAUDRATE = 9600;
...
if (BAUDRATE > 9600) begin
... E.g. use the parameter in your code like you might any general variable
... BAUDRATE is completely local to this module and this instance. You might
... have the same parameters in 3 other UART instances and they'd all be different
... values...
```

Now, your test12 has all kinds of settings required for it. Let's define a special module called **testparams** which specifies all these settings. It will itself be a module instantiated under the testbench:

```
module testparams;
defparam testbench.cpu.uart1.BAUDRATE = 19200;
defparam testbench.cpu.uart2.BAUDRATE = 9600;
defparam testbench.cpu.uart3.BAUDRATE = 9600;
defparam testbench.clockrate CLOCKRATE = 200; // Period in ns.
... etc ...
endmodule
```

The above module always has the same module name, but you would have many different filenames; one for each test. So, the above would be kept in test12_params.v. Your Makefile includes the appropriate params file given the desired make target. (BTW: You may run across this sort of technique by ASIC vendors who might have a module containing parameters for a memory model, or you might see this used to collect together a large number of system calls that turn off timing or warnings on particular troublesome nets, etc. etc.)

- Use **memory blocks**. Not as common a technique, but something to consider. Since Verilog has a very convenient syntax for declaring and loading memories, you can store your input data in a hex file and use $readmemh to read all the data in at once.
In your testbench:

```
module testbench;
...
reg [31:0] control[0:1023];
...
initial $readmemh ("control.hex", control);
...
endmodule
```

You could vary the filename using the previous techniques. The control.hex file is just a file of hex values for the parameters. Luckily, $readmemh allows embedded comments, so you can keep the file very readable:

```
A000 // Starting address to put boot code in
10 // Activate all ten input pulse sources
... etc...
```

Obviously, you are limitied to actual hex values with this approach. Note, of course, that you are free to mix and match all of these techniques!